SQL vs MongoDB

SQL	MongoDB
`INT`	`Int32`
`BIGINT`	`Int64 (Long)`
`FLOAT`	`Double`
`DECIMAL`	`Decimal128`
`VARCHAR`	`String`
`CHAR`	`String`
`TEXT`	`String`
`BOOLEAN`	`Boolean`
`DATE`	

`BOOLEAN`	`Boolean`
`DATE`	`Date`
`TIMESTAMP`	`Timestamp` / `Date`
`BLOB`	`Binary`
Table	Collection
Row	Document
Column	Field

MongoDB BSON Data Types

Upgr

Data Type	BSON Type Number	Size	Use
`String`	2	Variable	Name, Email, Address
`Int32`	16	4 bytes	Age, Marks
`Int64 (Long)`	18	8 bytes	Large IDs, Big Numbers
`Double`	1	8 bytes	Decimal approximation
`Decimal128`	19	16 bytes	Money, Salary
`Boolean`	8	1 byte	true / false
`Date`	9	8 bytes	DOB, CreatedAt
`Timestamp`	17	8 bytes	Replication, Internal
`ObjectId`	7	12 bytes	Default `_id`

`ObjectId`	7	12 bytes	Default `_id`
`Array`	4	Variable	Skills, Subjects
`Object (Document)`	3	Variable	Nested Documents
`Null`	10	-	Null values
`Binary`	5	Variable	Files, Images
`Regular Expression`	11	Variable	Regex
`JavaScript`	13	Variable	JS Code (Rare)
`MinKey`	-1	-	Internal
`MaxKey`	127	-	Internal

Mongoose crude

ery.js > main

```
const mongoose = require("mongoose");

async function main() {
    try {
        await mongoose.connect("mongodb://127.0.0.1:27017/mytest");
        console.log("Connected");
        console.log("Database:", mongoose.connection.name);
      // All Collections
const collections = await mongoose.connection.db.listCollections().toArray();
       // console.log(collections);
      collections.forEach((col) => {
    console.log(col.name);
});
mongoose.connection.close();
    } catch (err) {
        console.log(err);
    }
}
```

2. Database Delete

JavaScript

```javascript
const mongoose = require("mongoose");

async function main() {
    await mongoose.connect("mongodb://127.0.0.1:27017/mytest");

    await mongoose.connection.dropDatabase();

    console.log("Database Deleted");

    mongoose.connection.close();
}

main();
```

```javascript
const mongoose = require("mongoose");

async function main() {
    try {
        await mongoose.connect("mongodb://127.0.0.1:27017/mytest");
        console.log("Connected");

        const admin = mongoose.connection.db.admin();

        const dbs = await admin.listDatabases();

        dbs.databases.forEach((db) => {
            console.log(db.name);
        });

        mongoose.connection.close();
    } catch (err) {
        console.log(err);
    }
}

main();
```

3. Collection Rename

JavaScript

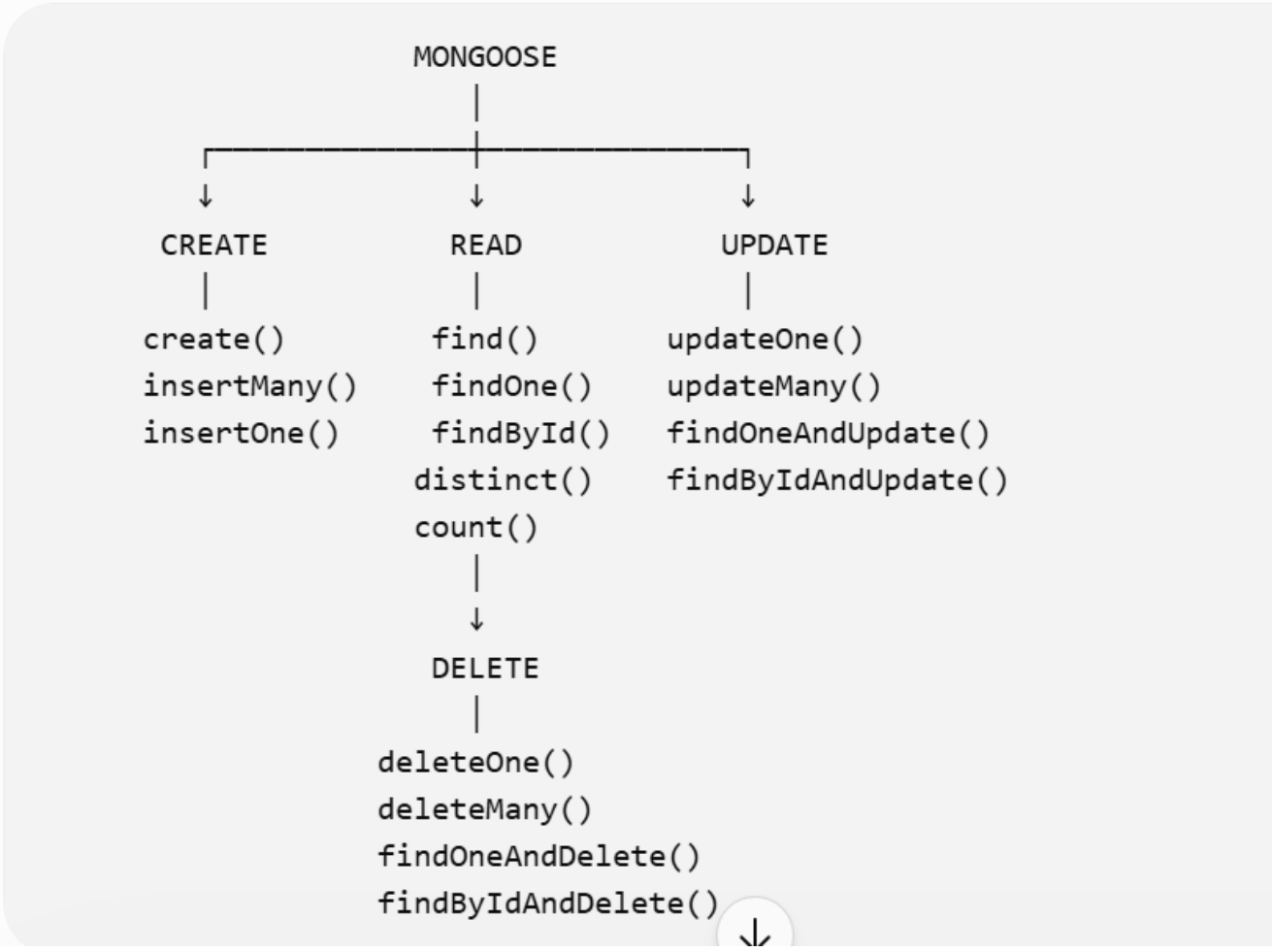
```javascript
const mongoose = require("mongoose");

async function main() {
    await mongoose.connect("mongodb://127.0.0.1:27017/mytest");

    await mongoose.connection.db
        .collection("users")
        .rename("students");

    console.log("Collection Renamed");

    mongoose.connection.close();
}

main();
```

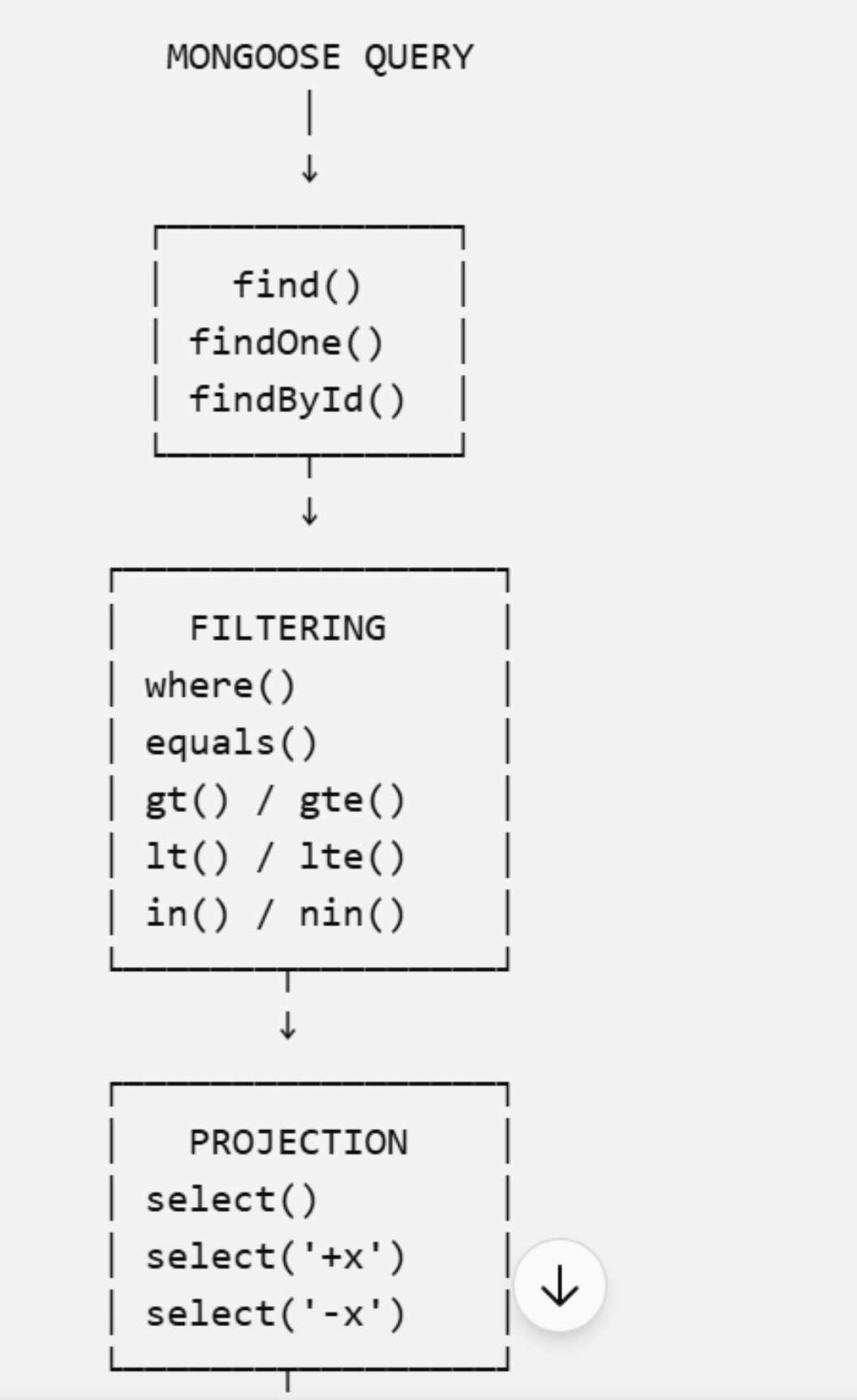
4. Collection Delete

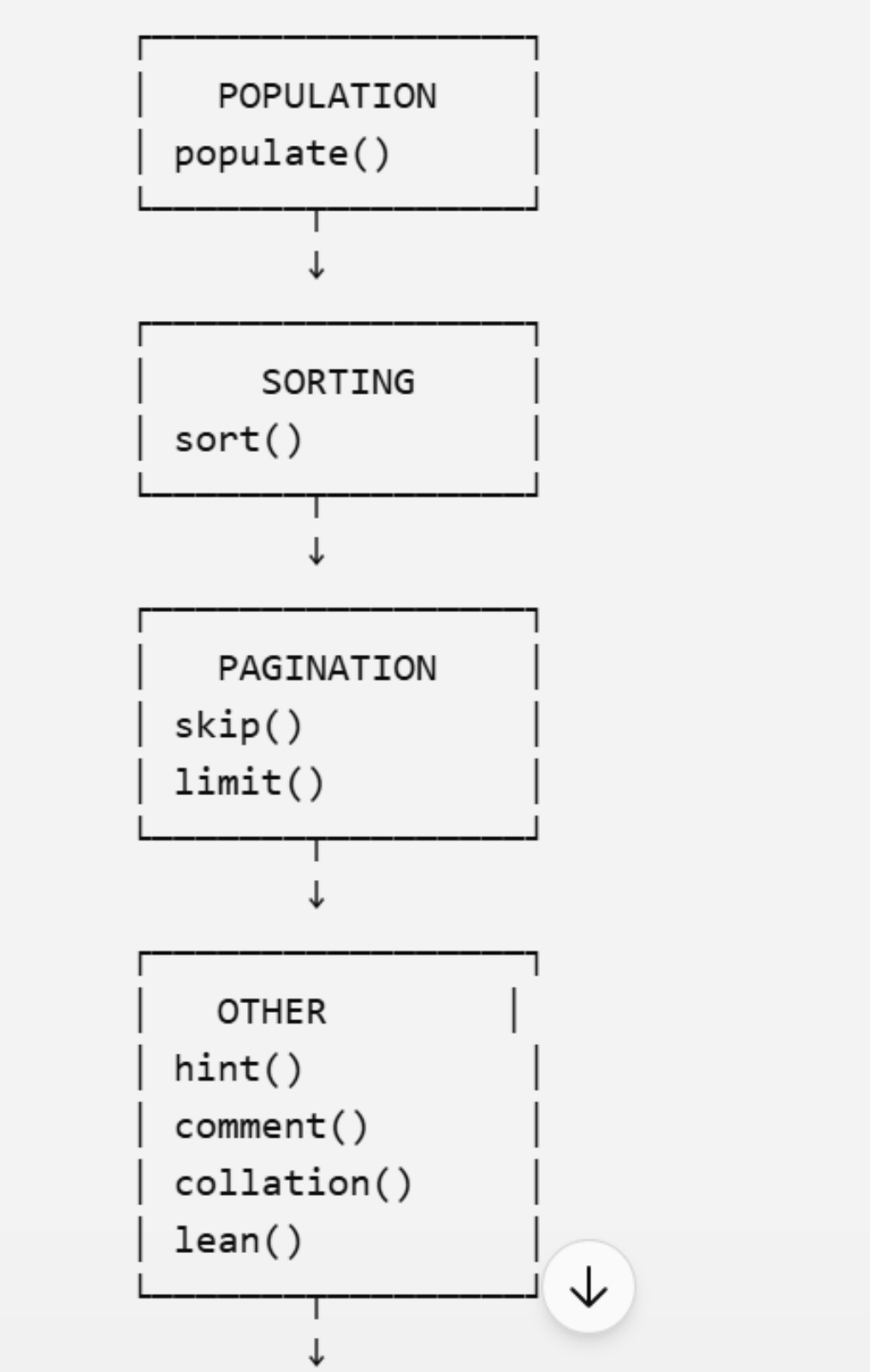
JavaScript

```javascript
const mongoose = require("mongoose");

async function main() {
    await mongoose.connect("mongodb://127.0.0.1:27017/mytest");

    await mongoose.connection.db
        .collection("users")
        .drop();

    console.log("Collection Deleted");

    mongoose.connection.close();
}

main();
```

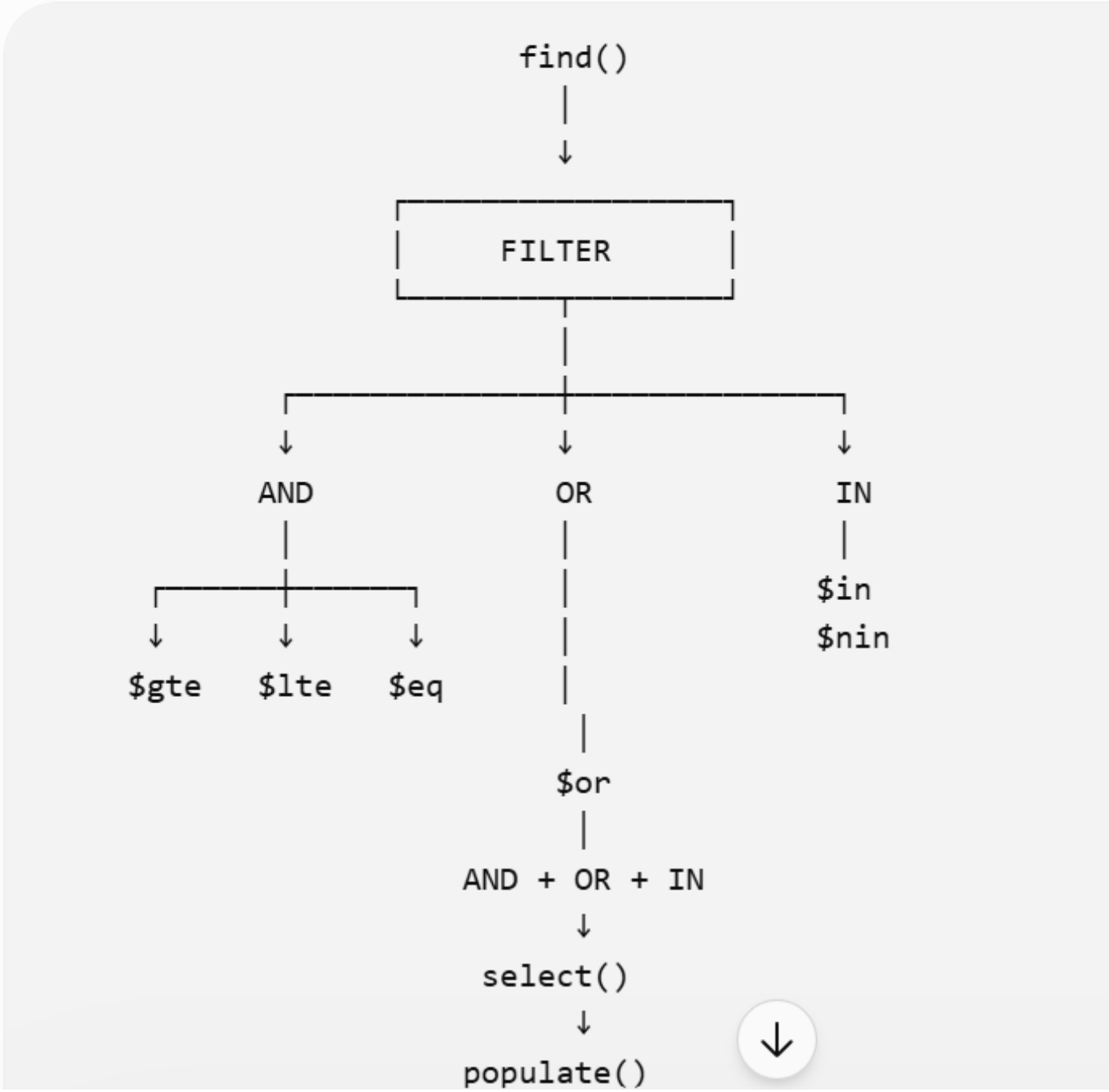
Query methods ko **CRUD-wise** yaad karna interview ke liye better hai:




MONGOOSE QUERY
find()
findOne()
findById()
FILTERING
where()
equals()
gt() / gte()
lt() / lte()
in() / nin()
PROJECTION
select()
select('+x')
select('-x')


POPULATION
populate()
SORTING
sort()
PAGINATION
skip()
limit()
OTHER
hint()
comment()
collation()
lean()

Final Mongoose FIND hierarchy



d10-306aec494223

```
        ↓
    select()
        ↓
   populate()
        ↓
     sort()
        ↓
     skip()
        ↓
    limit()
        ↓
     lean()
        ↓
     await
        ↓
    RESULT
```

Sabse important shortcut:

`find()` ke baad pehle **data filter karo** (`AND/OR/IN`), phir **fields choose** (`select`), phir **related data** (`populate`), phir **order** (`sort`), phir **pagination** (`skip/limit`), aur end me read performance ke liye `lean()` .

```javascript
const mongoose = require("mongoose");

const firstSchema = new mongoose.Schema(
    {},
    { strict: false }
);

const secondSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            unique: true
        }
    },
    { strict: false }
);


const firstModel = mongoose.model(
    "firstModel",
    firstSchema,
    "firstModel"
);
```

```
const secondModel = mongoose.model(
    "secondModel",
    secondSchema,
    "secondModel"
);


async function main() {

    try {

        console.time("api");

        await mongoose.connect(
            "mongodb://localhost:27017/test"
        );

        console.timeEnd("api");

        console.log("connect successful");
```

```
        const data = await firstModel
            .find({
                age: {
                    $lte: 55,
                    $gte: 23
                },
                city: "nalkheda"
            })
            .select("name age city")
            .sort({ age: -1 })
            .skip(2)
            .limit(5)
            .lean();

        console.log(data);

    }
    catch (err) {
        console.log(err);
    }

}
```

1. `where()` + `gte()` + `lte()`

Tumhari query ko is tarah bhi likh sakte ho:

JavaScript

```
const data = await firstModel
    .find()
    .where("age").gte(23).lte(55)
    .where("city").equals("nalkheda")
    .select("name age city")
    .sort({ age: -1 })
    .skip(2)
    .limit(5)
    .lean();
```

2. `populate()`

Agar ek collection me reference hai:

JavaScript

```
const data = await Student
    .find()
    .populate("department")
    .select("name age department")
    .sort({ age: -1 })
    .limit(5)
    .lean();
```

`populate()` ka use **reference wale docume ko automatically fetch karne** ke liye hota

-306aec494223

3. `exists()`

JavaScript

```
const data = await firstModel
    .find()
    .where("email").exists(true);
```

Meaning:

```
email field present hona chahiye
```

---

4. `in()`

JavaScript

```
const data = await firstModel
    .find()
    .where("city")
    .in(["Indore", "Bhopal", "Ujjain"
```

5. `regex`

Search ke liye:

```javascript
const data = await firstModel.find({
    name: {
        $regex: "^A",
        $options: "i"
    }
});
```

Meaning:

```
Name A se start ho
```

Agar unique values chahiye:

JavaScript

```
const cities = await firstModel.distinct("city");

console.log(cities);
```

Example:

```
["Indore", "Bhopal", "Ujjain", "Dewas"]
```

7. countDocuments()

Kitne documents match karte hain:

JavaScript

```
const count = await firstModel
    .countDocuments({
        city: "nalkheda"
    });

console.log(count);
```

JavaScript

```
const data = await firstModel
    .find({
        age: { $gte: 23 },

        $or: [
            { city: "Indore" },
            { city: "Bhopal" }
        ]
    })
    .select("name age city")
    .sort({ age: -1 })
    .skip(2)
    .limit(5)
    .lean();
```

7. AND + OR + IN — Advanced

Upg

Suppose question:

> Age 23–55 ke beech ho **AND** city Indore/Bhopal/Ujjain me ho **AND** marks 60 se zyada ho.

JavaScript

```
const data = await firstModel
    .find({
        age: {
            $gte: 23,
            $lte: 55
        },

        city: {
            $in: [
                "Indore",
                "Bhopal",
                "Ujjain"
            ]
        },
```

```
            ]
        },

        marks: {
            $gt: 60
        }
    })
    .select("name age city marks")
    .sort({ age: -1 })
    .skip(2)
    .limit(5)
    .lean();
```

Yahan top-level conditions automatically **AND** hain:

```
age 23-55
      AND
city IN [Indore, Bhopal, Ujjain]
      AND
marks > 60
```

Mongoose crude

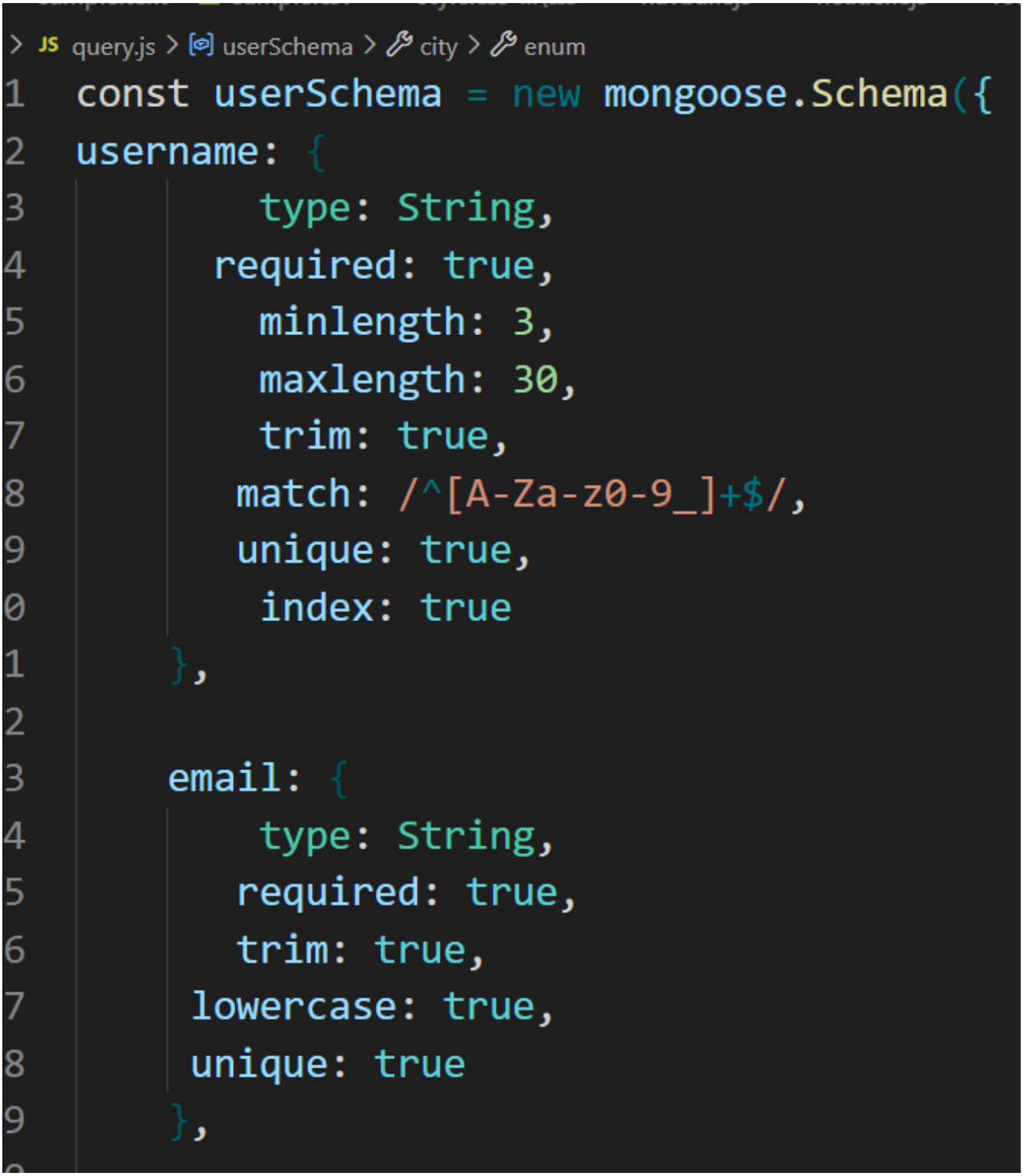
```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({

    phone: {
        type: String,
       required: true,
       unique: true,
       match: /^[789][0-9]{9}$/
    },

password: {
        type: String,
        required: true,
       minlength: 16,
      maxlength: 16,
match: /^(?=[A-Z])(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@#$])[A-Za-z0-9@#$]{16}$/
    },


    dob: {
        type: String,
        required: true,
match: /^(0[1-9]|[12][0-9]|3[01])\/(0[1-9]|1[0-2])\/[0-9]{4}$/
    }
});

const User = mongoose.model("User", userSchema);
```

JavaScript

```
async function main() {

    await mongoose.connect(
        "mongodb://localhost:27017/test"
    );

    try {

        const user = await User.create({

            phone: "9876543210",

            password: "Abcdefgh12345@#$",

            dob: "18/08/2002"

        });

        console.log(user);

    }
    catch (err) {
```

JavaScript

```
const users = await User.insertMany([

    {
        phone: "9876543210",
        password: "Abcdefgh12345@#$",
        dob: "18/08/2002"
    },

    {
        phone: "8765432109",
        password: "Xabcdef1234567@$",
        dob: "25/12/2001"
    },

    {
        phone: "7654321098",
        password: "Pqwerty1234567@#",
        dob: "05/01/2003"
    }

]);

console.log(users);
```


query.js > userSchema > city > enum
const userSchema = new mongoose.Schema({
username: {
type: String,
required: true,
minlength: 3,
maxlength: 30,
trim: true,
match: /^[A-Za-z0-9_]+$/,
unique: true,
index: true
},
email: {
type: String,
required: true,
trim: true,
lowercase: true,
unique: true
},

```
    },

    password: {
        type: String,
       required: true,
       minlength: 16,
        maxlength: 16,
match: /^(?=[A-Z])(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@#$])[A-Za-z0-9@#$]{16}$/
    },

    city: {
        type: String,
       default: "Indore",
enum: ["Indore","Bhopal","Ujjain","Dewas"]
    }

});
```

```
VALIDATION
├── required
├── minlength
├── maxlength
├── enum
├── match
└── validate

TRANSFORMATION
├── trim
├── lowercase
└── uppercase

DATABASE / INDEX
├── unique
└── index

DEFAULT
└── default
```

Ye distinction interview me "**Mongoose schema ke String type ke options kaun-kaun se hain?**" wale question me kaafi useful hai.

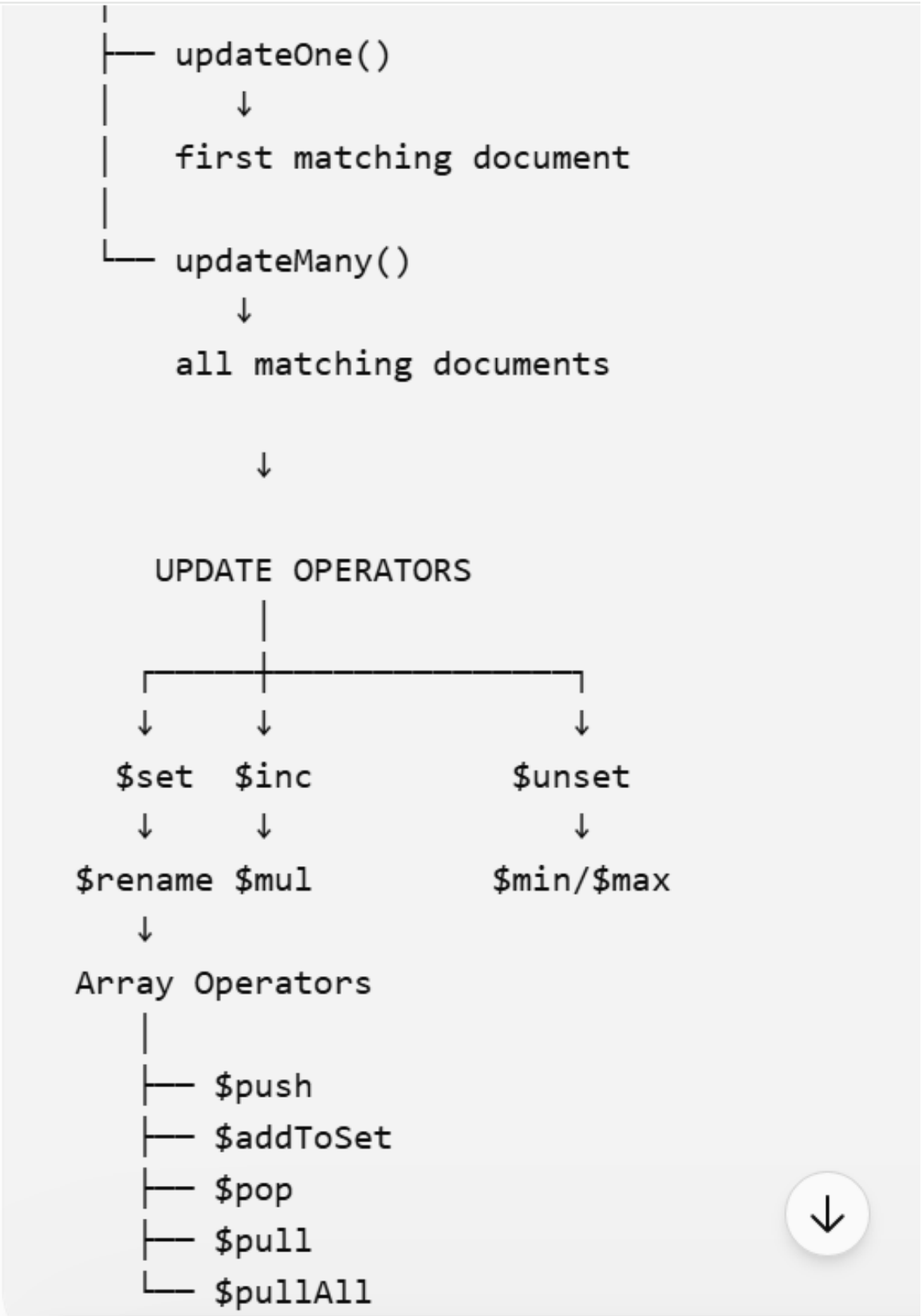
#	Option	Purpose	Example	Result
1	`type`	Data type define karta hai	`type: String`	`"Tarun"`
2	`required`	Field compulsory banata hai	`required: true`	Value na dene par validation error
3	`minlength`	Minimum string length	`minlength: 3`	`"Ta"` ❌
4	`maxlength`	Maximum string length	`maxlength: 50`	51 chars ❌
5	`trim`	Beginning/end ke spaces remove karta hai	`trim: true`	`" Tarun "` → `"Tarun"`
6	`lowercase`	Value ko lowercase karta hai	`lowercase: true`	`"TARUN"` → `"tarun"`
7	`uppercase`	Value ko uppercase karta hai	`uppercase: true`	`"tarun"` → `"TARUN"`
8	`enum`	Sirf given values allow karta hai	`enum: ["Male","Female"]`	`"Other"` ❌
8	`enum`	Sirf given values allow karta hai	`enum: ["Male","Female"]`	`"Other"` ❌
9	`match`	Regex pattern validate karta hai	`match: /^[A-Za-z]+$/`	`"Tarun123"` ❌
10	`validate`	Custom validation condition	`validate: fn`	Apni condition laga sakte ho
11	`unique`	Duplicate value ke liye unique index	`unique: true`	Same email duplicate ❌
12	`index`	Field par index create karta hai	`index: true`	Searching/querying faster ho sakti hai
13	`default`	Value na dene par default value	`default: "Unknown"`	Missing → `"Unknown"`

14	`alias`	Field ka alternate name define karta hai	`alias: "userName"`	`name` ko `userName` se access
15	`immutable`	Set hone ke baad change prevent karta hai	`immutable: true`	Update me change nahi hoga
16	`select`	Query result me field by default include/exclude	`select: false`	Password normally return nahi hoga
17	`get`	Read karte waqt value transform	`get: v => v.trim()`	Output transform
18	`set`	Save karte waqt value transform	`set: v => v.trim()`	Save se pehle transform

JavaScript

```javascript
const userSchema = new mongoose.Schema({

    name: {
        type: String,
        required: true,
        minlength: 3,
        maxlength: 50,
        trim: true,
        lowercase: true,
        uppercase: false,
        enum: ["Rahul", "Amit", "Tarun"],
        match: /^[A-Za-z ]+$/,
        unique: true,
        index: true,
        default: "Unknown"
    }

});
```

306aec494223



JavaScript

```
await Model.updateOne(
    { filter },
    { update }
);
```

Example

JavaScript

```
const result = await firstModel.updateOne(
    { name: "Rahul" },
    {
        $set: {
            age: 25,
            city: "Indore"
        }
    }
);

console.log(result);
```

2. updateMany()

Saare matching documents ko update karta hai.

JavaScript

```
const result = await firstModel.updateMany(
    { city: "Indore" },
    {
        $set: {
            status: "active"
        }
    }
);

console.log(result);
```

Meaning:

```
city = Indore
     ↓
ALL matching documents
     ↓
status = active
```

3. Update operators

`$set` ke alawa bahut operators hain.

Operator	Purpose	Example
`$set`	Value set/update	`$set: {age: 25}`
`$unset`	Field remove	`$unset: {age: 1}`
`$inc`	Number increase/decrease	`$inc: {age: 1}`
`$mul`	Number multiply	`$mul: {salary: 2}`
`$min`	Smaller value preserve	`$min: {marks: 50}`
`$max`	Larger value preserve	`$max: {marks: 90}`
`$rename`	Field rename	`$rename: {name:"fullName"}`
`$push`	Array me add	`$push: {skills:"JS"}`
`$addToSet`	Array me unique value add	`$addToSet:{skills:"JS"}`
`$addToSet`	Array me unique value add	`$addToSet:{skills:"JS"}`
`$pop`	Array ka first/last remove	`$pop:{skills:1}`
`$pull`	Array se matching value remove	`$pull:{skills:"Java"}`
`$pullAll`	Multiple array values remove	`$pullAll:{skills:["Java","C"]}`

5. `$inc`

Age 1 increase:

JavaScript

```
await firstModel.updateMany(
    { city: "Indore" },
    {
        $inc: {
            age: 1
        }
    }
);
```

Agar:

```
Rahul → 23
Amit  → 25
Ravi  → 30
```

6. `$unset`

Field delete karna:

JavaScript

```
await firstModel.updateOne(
    { name: "Rahul" },
    {
        $unset: {
            city: 1
        }
    }
);
```

`city` field remove ho jayegi.

7. `$rename`

JavaScript

```
await firstModel.updateMany(
    {},
    {
        $rename: {
            "name": "fullName"
        }
    }
);
```

Before:

JavaScript

```
{
    name: "Rahul"
}
```

8. `$min`

Suppose current marks:

```
80
```

JavaScript

```
await firstModel.updateOne(
    { name: "Rahul" },
    {
        $min: {
            marks: 70
        }
    }
);
```

Result:

```
70
```

10. Array `$push`

Suppose:

JavaScript

```
{
    name: "Rahul",
    skills: ["C++", "SQL"]
}
```

JavaScript

```
await firstModel.updateOne(
    { name: "Rahul" },
    {
        $push: {
            skills: "JavaScript"
        }
    }
);
```

Result:

13. Multiple operators ek saath

JavaScript

```
await firstModel.updateOne(
    { name: "Rahul" },
    {
        $set: {
            city: "Bhopal"
        },

        $inc: {
            age: 1
        },

        $unset: {
            temporaryField: 1
        }
    }
);
```

Ek hi update me:

1. `deleteOne()`

Condition se match hone wala first document delete karta hai.

Syntax

JavaScript

```
await Model.deleteOne(
    { condition }
);
```

Example

JavaScript

```
const result = await firstModel.deleteOne({
    name: "Rahul"
});

console.log(result);
```

2. `deleteMany()`

Condition se match hone wale saare documents delete karta hai.

JavaScript

```
const result = await firstModel.deleteMany({
    city: "Indore"
});

console.log(result);
```

Agar 20 students ki city `"Indore"` hai:

```
20 matching
   ↓
deleteMany()
   ↓
20 deleted
```

3. Filter ke saath delete

Age 50 se greater

JavaScript

```
await firstModel.deleteMany({
    age: {
        $gt: 50
    }
});
```

Age 23–30

JavaScript

```
await firstModel.deleteMany({
    age: {
        $gte: 23,
        $lte: 30
    }
});
```

4. AND condition

JavaScript

```
await firstModel.deleteMany({
    age: {
        $gte: 23,
        $lte: 30
    },
    city: "Indore"
});
```

Meaning:

```
age >= 23
AND
age <= 30
AND
city = Indore
```

5. OR condition

JavaScript

```
await firstModel.deleteMany({
    $or: [
        { city: "Indore" },
        { city: "Bhopal" }
    ]
});
```

Meaning:

```
city = Indore
OR
city = Bhopal
```

7. `findOneAndDelete()`

Agar **document delete karna hai aur deleted document bhi return chahiye**, to:

JavaScript

```
const deletedUser = await firstModel.findOneAndDelete({
    name: "Rahul"
});

console.log(deletedUser);
```

Yahan:

```
find
 ↓
delete
 ↓
deleted document return
```

10. `deleteOne()` vs `deleteMany()`

Point	`deleteOne()`	`deleteMany()`
Documents	First matching	All matching
Filter	`{}`	`{}`
Return	Delete result	Delete result
`deletedCount`	Usually `0` or `1`	`0` to many
Use	Single document	Bulk delete

Example:

Specific cities

JavaScript

```
await firstModel.deleteMany({
    city: {
        $in: ["Indore", "Bhopal", "Ujjain"]
    }
});
```

Meaning:

```
city = Indore
OR
city = Bhopal
OR
city = Ujjain
```

`populate()` ki zarurat kyu padi.

1. Sabse pehle: Relationship kya hai?

Maan lo hamare application me **User** aur **Post** hain.

Ek user bahut saare posts bana sakta hai.

```
User
  ↓
Post
  ↓
Post
  ↓
Post
```

Example:

-d097aeba1769

User collection

_id	name
U101	Tarun
U102	Rahul

Post collection

_id	title	user
P1	JavaScript	U101
P2	MongoDB	U101
P3	React	U102

Yahan `Post.user` me **User ki ID** rakhi hai.

2. Post me User ki ID hi kyu rakhi?

Hum Post ke andar pura user object store kar sakte the:

JavaScript

```
{
  title: "MongoDB",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com",
    age: 22
  }
}
```

Lekin ye achha design nahi hai.

Kyun?

Maan lo Tarun ke **1000 posts** hain.

Maan lo Tarun ke **1000 posts** hain.

To 1000 documents me:

```
name: Tarun
email: ...
age: ...
```

repeat hoga.

Agar Tarun apna email change kare:

```
tarun@gmail.com
        ↓
new@gmail.com
```

to potentially bahut saare documents update karne padenge.

Isliye hum sirf **User ki ID** store karte hain:

JavaScript

```
{
  title: "MongoDB",
  user: "U101"
}
```

Aur User ki actual information User collection me rahegi.

3. Ab problem kya aayi?

Ab hum Post nikalte hain:

JavaScript

```
const posts = await Post.find();
```

Result:

JavaScript

```
[
  {
    title: "JavaScript",
    user: "U101"
  },
  {
    title: "MongoDB",
    user: "U101"
  }
]
```

Hume pata hai:

Hume pata hai:

```
U101 = Tarun
```

Lekin response me sirf:

```
user: U101
```

aa raha hai.

Frontend ko chahiye:

JavaScript

```
{
  title: "MongoDB",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

To ab question hai:

> **U101 ki actual details kaise laayenge?**

Yahin `populate()` kaam aata hai.

---

4. Populate kya karta hai?

Simple definition:

> **`populate()` kisi document me stored reference ID ko use karke related document ki actual information fetch karta hai.**

Example:

JavaScript

```
const posts = await Post.find().populate("user");
```

Mongoose dekhega:

```javascript
user: "U101"
```

Aur schema me dekhega:

```javascript
ref: "User"
```

To Mongoose samjhega:

```
U101
 ↓
User collection
 ↓
U101 wala document
```

Aur result me user ki details laga dega.

5. Without populate vs With populate

Without `populate()`

```javascript
const posts = await Post.find();
```

Result:

```javascript
{
  title: "MongoDB",
  user: "U101"
}
```

Sirf ID mili.

With `populate()`

JavaScript

```
const posts = await Post.find().populate("user");
```

Result:

JavaScript

```
{
  title: "MongoDB",
  user: {
    _id: "U101",
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

Ab ID ki jagah related User document mil gaya.

6. Populate actually kaise pata karta hai ki U101 User hai?

Ye bahut important hai.

Schema me hum likhte hain:

JavaScript

```
const postSchema = new mongoose.Schema({
  title: String,

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});
```

Yahan 3 important cheezein hain:

`type`

```javascript
type: mongoose.Schema.Types.ObjectId
```

Matlab:

> `user` field me kisi document ki ID store hogi.

`ref`

```javascript
ref: "User"
```

Matlab:

> Ye ID **User model** ke document ko refer karti hai.

Ab jab:

JavaScript

```
.populate("user")
```

likha:

Mongoose samajh gaya:

```
Post ka "user" field
        ↓
isme User ki ObjectId hai
        ↓
User model me jao
        ↓
matching document lao
```

7. Real-life example

Maan lo tumhare paas ek **college database** hai.

Student

JavaScript

```
{
  _id: 101,
  name: "Tarun"
}
```

Course

JavaScript

```
{
  _id: 501,
  name: "MongoDB",
  student: 101
}
```

Course me humne:

```
student = 101
```

store kiya.

Lekin agar API response me chahiye:

JavaScript

```
{
  name: "MongoDB",
  student: {
    name: "Tarun"
  }
}
```

to:

to:

```javascript
Course.find().populate("student")
```

use kar sakte hain.

---

8. SQL JOIN se relation

Agar tum SQL jaante ho, to aur easy hai.

SQL me:

```sql
SELECT *
FROM posts
JOIN users
ON posts.user_id = users.id;
```

MongoDB + Mongoose me roughly:

8. SQL JOIN se relation

Agar tum SQL jaante ho, to aur easy hai.

SQL me:

SQL

```sql
SELECT *
FROM posts
JOIN users
ON posts.user_id = users.id;
```

MongoDB + Mongoose me roughly:

JavaScript

```javascript
Post.find().populate("user");
```

Dono ka basic purpose:

```
Related data ko saath me lana
```

Hai.

Lekin technically `populate()` SQL JOIN exactly nahi hai. Mongoose related documents fetch karke references ko replace karta hai.

---

9. Populate ki zarurat kyu padi?

Main reason hai **data ko duplicate na karna + relationship maintain karna**.

Suppose Tarun ke 100 posts hain.

Bad approach

Har Post me:

JavaScript

```
{
  title: "Post 1",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

```
{
  title: "Post 2",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

JavaScript

```
{
  title: "Post 3",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

User information repeat ho rahi hai.

Better approach

Post me:

JavaScript

```
{
  title: "Post 1",
  user: ObjectId("...")
}
```

JavaScript

```
{
  title: "Post 2",
  user: ObjectId("...")
}
```

JavaScript

```
{
  title: "Post 3",
  user: ObjectId("...")
}
```

Aur User ki information:

```
{
  _id: ObjectId("..."),
  name: "Tarun",
  email: "tarun@gmail.com"
}
```

Ek hi jagah stored hai.

Jab actual user details chahiye:

```
.populate("user")
```

10. Ek complete example

User Model

JavaScript

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});

const User = mongoose.model("User", userSchema);
```

Post Model

JavaScript

```
const postSchema = new mongoose.Schema({
  title: String,

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
```

JavaScript

```
const postSchema = new mongoose.Schema({
  title: String,

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
});

const Post = mongoose.model("Post", postSchema);
```

Data

User:

JavaScript

```
{
  _id: ObjectId("111"),
  name: "Tarun",
  email: "tarun@gmail.com"
}
```

Post:

```javascript
{
  _id: ObjectId("222"),
  title: "MongoDB Populate",
  user: ObjectId("111")
}
```

Query

```javascript
const post = await Post
  .find()
  .populate("user");
```

Result

```javascript
{
  _id: "222",
  title: "MongoDB Populate",

  user: {
    _id: "111",
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

11. Populate me `"user"` hi kyu likha?

Schema:

```JavaScript
user: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User"
}
```

Query:

```JavaScript
.populate("user")
```

Yahan `"user"` **Post ke field ka naam** hai.

Agar schema hota:

```javascript
author: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User"
}
```

to:

```javascript
.populate("author")
```

likhte.

So:

```
Schema field
     ↓
populate("same field")
```

12. Specific fields bhi la sakte ho

Maan lo User me:

```javascript
{
  name: "Tarun",
  email: "tarun@gmail.com",
  password: "123456"
}
```

Password nahi chahiye.

To:

```javascript
Post.find().populate("user", "name email");
```

Result:

Result:

```javascript
{
  title: "MongoDB",
  user: {
    name: "Tarun",
    email: "tarun@gmail.com"
  }
}
```

Ye **security ke liye bhi important** hai.